

Improving Data Curation of Software Vulnerability Patches through Uncertainty Quantification

Hui Chen, Yunhua Zhao, Kostadin Damevski

Abstract—The changesets (or patches) that fix open source software vulnerabilities form critical datasets for various machine learning security-enhancing applications, such as automated vulnerability patching and silent fix detection. These patch datasets are derived from extensive collections of historical vulnerability fixes, maintained in databases like the Common Vulnerabilities and Exposures list and the National Vulnerability Database. However, since these databases focus on rapid notification to the security community, they contain significant inaccuracies and omissions that have a negative impact on downstream software security quality assurance tasks.

In this paper, we propose an approach employing Uncertainty Quantification (UQ) to curate datasets of publicly-available software vulnerability patches. Our methodology leverages machine learning models that incorporate UQ to differentiate between patches based on their potential utility. We begin by evaluating a number of popular UQ techniques, including Vanilla, Monte Carlo Dropout, and Model Ensemble, as well as homoscedastic and heteroscedastic models of noise. Our findings indicate that Model Ensemble and heteroscedastic models are the best choices for vulnerability patch datasets. Based on these UQ modeling choices, we propose a heuristic that uses UQ to filter out lower quality instances and select instances with high utility value from the vulnerability dataset. Using our approach, we observe an improvement in predictive performance and a significant reduction of model training time (i.e., energy consumption) for a state-of-the-art vulnerability prediction model.

Index Terms—Software Vulnerability, Vulnerability Patch, Data Quality, Machine Learning

I. INTRODUCTION

When software vulnerabilities are reported and repaired in open-source software, the fix is recorded as a changeset (a commit or a group of semantically-related commits) in the software repository. Datasets of historic changesets (or patches) to vulnerabilities serve multiple purposes in preventing and mitigating the effects of future software vulnerabilities. They are used for automatic vulnerability patching techniques (e.g., hot-patching) [1], [2], for automatically detecting a silent fix that has not yet been publicized [3], [4], and for vulnerable code clone detection [5], [6]. Vulnerability patch datasets can also be used to extract vulnerable code to construct machine learning-based vulnerability prediction models, which have recently been showing promising results [7]–[11].

Large collections of vulnerability patches form the foundation for the aforementioned efforts. Government and industry have initiated concerted efforts to disseminate information

about known security vulnerabilities. These include MITRE’s Common Vulnerabilities and Exposures (CVE) list, the National Vulnerability Database (NVD), the Snyk Vulnerability Database, and GitHub Security Advisories. However, these efforts primarily focus on rapidly sharing details about newly found vulnerabilities to mitigate their impacts, differing from the needs of researchers who seek large datasets for building machine learning models or gathering empirical evidence to advance security-related software quality assurance [12].

To close this gap, research groups have curated historic vulnerability databases using: 1) manual validation of each patch for quality; 2) automated approaches based on data selection heuristics; or 3) machine learning-based approaches trained on small manually-curated datasets. These projects face two primary challenges. First, they rely on publicly disclosed vulnerability information, such as that from the NVD, which recent research shows is often missing important information or inaccurate [13], [14]. For example, Tan et al. found that out of 6,628 CVEs, only 66.57% contained references to patches, and of these, 32.79% were incorrect [13]. Heuristics and machine learning models do not eliminate the errors present in these datasets and could even exacerbate them. Second, manual review of vulnerability patches to ensure high quality results in very limited dataset sizes. This effort is further complicated by the need for some familiarity with different software projects to validate each patch. Despite these costly manual efforts, the data can still suffer from significant quality issues [15].

To address concerns about the quantity and quality of historic software vulnerability patch datasets, data curation must include the following characteristics: 1) automatic identification of patches to ensure sufficient data quantity, and 2) mitigation of the effects of inaccurately linked patches to maintain data quality for downstream uses, such as machine learning-based vulnerability prediction.

Aside from these, prior works mostly focus on patch data quality when the patches are curated or removed from an existing dataset [15], although the improved patch datasets are often evaluated via intended down-stream applications [13], [14]. We argue that data quality and data usefulness (i.e., the data utility value) are two related but distinct concepts and the data quality and the data usefulness exist in a spectrum. Prior works often lack of systematic or algorithmic methods to assess patch quality and usefulness. Furthermore, existing efforts typically focus on dataset cleaning, aiming to address technical errors and inconsistencies. As we demonstrate, our data curation approach instead selects data points that maximize utility, ensuring that the resulting dataset is optimized for improving model performance and training efficiency.

H. Chen and Y. Zhao are with the Department of Computer & Information Science, CUNY Brooklyn College and with the Department of Computer Science, CUNY Graduate Center, E-mails: huichen@ieee.org, yzhao5@gradcenter.cuny.edu

K. Damevski is with the Department of Computer Science, Virginia Commonwealth University, E-mail: {kdamevski}@vcu.edu

TABLE I
VULNERABILITY PATCH DATASETS ORGANIZED ACCORDING TO
COLLECTION APPROACH.

Dataset	Characteristics			Collection Appr.		
	lang.	# proj.	# patch	H1	H2	M
SecBench [22]	> 1	114	676	✓		
Lin et al. [23]	C/C++	9	1471	✓	✓	
VulData7 [24]	> 1	4	1600	✓	✓	
VulnCatcher [25]	C/C++	3	2879	✓	✓	
SecretPatch [26]	C/C++	898	1636		✓	
Big-Vul [27]	C/C++	348	10547		✓	
Sec. Patches [28]	C/C++	1339	5942		✓	
Riom et al. [29]	C/C++	50	470		✓	
CVEfixes [30]	> 1	1754	5495		✓	
VulCurator [31]	Python	1	290		✓	
VulnCode-DB [32]	> 1	-	3681		✓	
SAP [33]	Java	205	1282		✓	✓
VCMATCH [14]	C/C++	10	1669		✓	✓

H1: The CVE-IDs are mentioned in a commit message in the project repository. **H2:** A URL to the patch is available in the NVD listing for the CVE. **M:** Patches are manually validated.

Uncertainty Quantification (UQ) can ascertain a model’s acquisition of knowledge from vulnerability patch data and control the noise and error in the vulnerability patch dataset. Building on recent research on UQ in machine learning [16]–[20], this paper proposes a technique for integrating uncertainty quantification into software vulnerability patch data and machine learning-based models, along with a method to rank patches based on their quality and utility value. More specifically, we answer the following research questions:

RQ1. What UQ techniques are capable of assessing data quality and usefulness changes in software vulnerability patch datasets?

To address this question, we compare various UQ techniques combining two data distribution modeling approaches, homoscedastic and heteroscedastic [17], [21], with three UQ estimation methods: Vanilla, Monte Carlo Dropout, and Model Ensemble [16], [18], [19]. Our evaluation indicates that prediction confidence from the Vanilla approach alone is insufficient as a UQ measure. Additional uncertainty measures from Monte Carlo Dropout and Model Ensemble reveal higher uncertainty as the quality of vulnerability patches declines. Moreover, UQ models incorporating noise distributions, such as heteroscedastic, respond better to changes in data quality and usefulness in a software vulnerability dataset. An UQ model that disentangles epistemic and aleatoric uncertainties is particularly useful. Epistemic uncertainty can serve as a proxy for the usefulness of security patches while aleatoric uncertainty the quality or the noise level of the patches.

RQ2. Can we use UQ to improve automatically curated vulnerability patches by selecting high quality and highly usable security patches?

We propose an algorithm to select software vulnerability patches based on epistemic and aleatoric uncertainty. The selected patches are likely to have the highest quality and highest utility values. The approach can improve the quality of software patch datasets like those listed in Table I. More importantly, It can positively impact downstream machine

learning applications, such as serving the datasets as training instances for automatic vulnerability patching techniques [1], [2], for automatically detecting silent fixes that have not yet been publicized [3], [4], and for vulnerable code clone detection [5], [6]. To exhibit these, we conduct an experiment that indicates that our approach can select security patches to improve the predictive performance and computational time of software vulnerability prediction, a representative downstream use case of software vulnerability patch data.

Answering these two RQs leads to the following contributions:

- 1) An algorithm for automated and systematic curation of vulnerability patches based on UQ, which represents a first attempt to use both epistemic uncertainty and aleatoric uncertainty to inform the usefulness and the quality of security patches.
- 2) Empirical evidence that informs the design of the UQ-based vulnerability patch curation algorithm.
- 3) Statistically significant improvement of predictive performance and reduction of computational cost of a state-of-the-art software vulnerability prediction model.

Thus, our approach: 1) has practical implications for reducing training time and improving prediction accuracy across various applications involving pre-collected vulnerability data; 2) can expand existing datasets by curating high-quality, high-utility data instances from open-source projects; and 3) encourages further, much-needed research on the application of UQ to this problem.

Replication package. The replication package and supplementary material are available at <https://github.com/huichen-cs/vuldatapublic>.

II. ALEATORIC AND EPISTEMIC UNCERTAINTY IN SOFTWARE VULNERABILITY PATCH DATA CURATION

As shown in Table I, numerous efforts have been made to curate software vulnerability patch data. The primary approaches rely on heuristics like searching for CVE-IDs in commit messages and examining URLs provided in the NVD listing for CVEs (i.e., H1 and H2 in Table I), which are inherently noisy and incomplete. For instance, many links gathered through H2 do not point to the actual patch but rather to related code, while CVE-IDs in commits for H1 are infrequent and can be part of tangled commits that include changes unrelated to patching the vulnerability. Ensuring the high quality of vulnerability patch datasets, therefore, requires significant manual effort. This is reflected in the relatively small size of the manually validated datasets in Table I. For example, the SAP dataset [33] contains 1,282 patches manually curated by teams of developers at SAP through their daily work. Such manual data curation approaches face two main challenges. Firstly, they are difficult to scale due to the extensive manual inspection required and the limited availability of software security expertise. Secondly, they cannot address the issue of missing or broken links between CVEs and patches in the NVD.

Recently, machine learning-based approaches have emerged. In ML-based approaches, researchers manually curate an

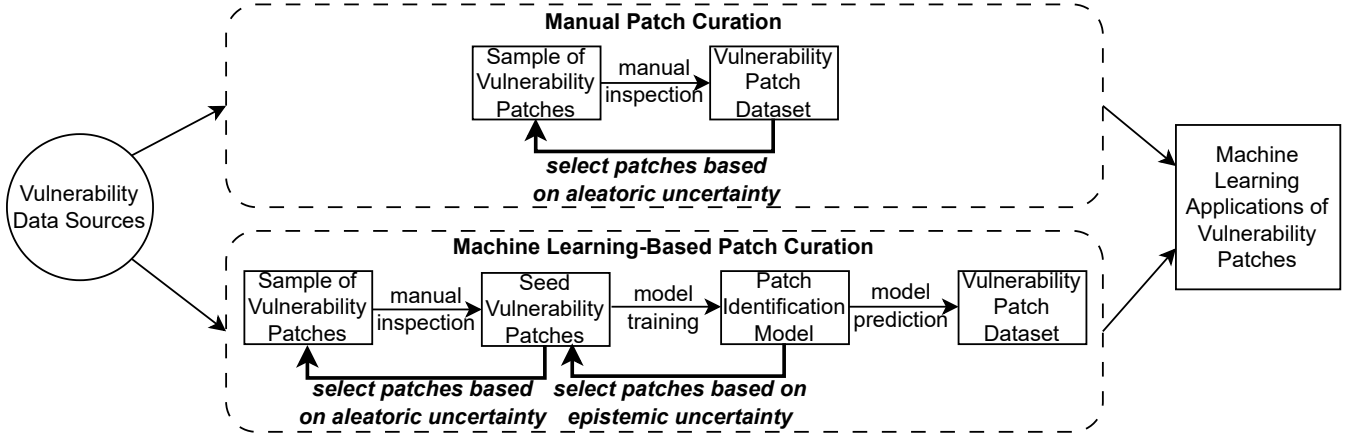


Fig. 1. Sources of aleatoric and epistemic uncertainty in manual and machine learning-based vulnerability patch identification.

initial high-quality set of “seed” vulnerability patches and use them to train a machine learning model, which then automatically identifies vulnerability patches [3], [4], [13], [14], [34]–[37]. Thus, ML-based approaches reduce the manual effort to curate larger datasets; however, they still suffer from two shortcomings: whether the instances curated are of low-quality or whether the instances contribute to improving down-stream applications, such as vulnerability prediction. Motivated by recent advances in uncertainty quantification in ML, we propose a data curation framework that addresses these two shortcomings as illustrated in Figure 1, which depicts the conceptual framework of the proposed approach.

Researchers distinguish between two types of uncertainty: aleatoric and epistemic uncertainty [17]. Aleatoric uncertainty arises from the data generation and curation process, which encompasses factors such as noise in the data and the representation of data features, both of which influence training data quality [38]. Conversely, epistemic uncertainty arises from inadequate modeling [39], i.e., uncertainty in the model’s parameters and structures [17], indicating the model’s lack of knowledge about the problem space.

Due to the coexistence of manual and ML-based approaches in vulnerability patch dataset curation, both aleatoric and epistemic uncertainties are present in patch curation [16], [17], [40]. Aleatoric uncertainty is inherently part of manual data curation, as evidenced by the pervasive errors in the NVD observed by Tan et al. [13]. It also arises in ML-based vulnerability patch curation, which employs a machine learning model based on features that determine the relationship between a patch and a vulnerability (e.g., using textual similarity between the commit log and the vulnerability description). There are two primary sources of aleatoric uncertainty in this process. First, the features may not capture all relevant information concerning the origination of software vulnerability patches. Second, the curation process itself can introduce errors and noise. For example, ML-based approaches like Patchscout [13] and VCMATCH [14] identify certain feature elements, such as file locations in NVD records and function locations in committed code, using regular expression patterns manually derived from a small sample set, which can omit some feature

elements and also introduce false positives, as illustrated in Figure 1.

Epistemic uncertainty is only present in ML-based approaches and arises from the model’s lack of knowledge about the problem. It can manifest as variance in the model’s predictions using the same training dataset. For example, different training runs of a deep neural network may yield different model weights, leading to varied predictions, which illustrates epistemic uncertainty. It is anticipated that adding more training data can eventually constrain the model weights, thereby reducing epistemic uncertainty [40]–[43]. However, this process can become computationally expensive if more training data instances are added indiscriminately, and there is no assurance of improved predictive performance if the added training data instances are of low quality or do not provide meaningful information to enhance the model.

Thus, the proposed ML-based software vulnerability patch data curation process aims to address both informativeness and quality issues by measuring both epistemic and aleatoric uncertainties. This dual uncertainty measurement, along with our EHALL heuristic, defined in Section IV, aims to enhance the efficacy of ML-based vulnerability data curation.

III. RQ1: WHAT UQ TECHNIQUES ARE CAPABLE OF ASSESSING DATA QUALITY AND USEFULNESS CHANGES IN SOFTWARE VULNERABILITY PATCH DATASETS?

Researchers in engineering and scientific domains increasingly use uncertainty quantification to address measurement errors and model uncertainties [43], [44]. While UQ studies provide numerous approaches to measure uncertainty, they also reveal that specific datasets and data types introduce domain-specific challenges and there is no one-fits-all uncertainty estimate [45], [46]. For instance, recent studies emphasize the presence of evolving distributions, noise, and the need for domain-specific UQ methods tailored to structured representations and unique patterns in source code data [47], [48]. In this first RQ, we aim to identify the most suitable UQ approximation techniques for addressing the unique noise and challenges associated with software vulnerability data curation. Applying UQ to vulnerability datasets is particularly

challenging because the domain of vulnerability patches and descriptions is unique and therefore it is not possible to determine in advance which UQ method will perform best. Vulnerability patches listed in public datasets vary significantly in quality and utility, necessitating precise UQ measurements to account for these variations.

There are numerous approaches for quantifying epistemic and aleatoric uncertainty. Our objective is to identify suitable UQ approaches for patch data curation, focusing on neural network-based models, as they dominate ML-based curation tasks. We prioritize methods that integrate seamlessly without architectural modifications and can handle data distribution shifts common in vulnerability patches [48].

Probabilistic UQ methods are categorized into Bayesian and Frequentist approaches [39]. While conformal prediction, a popular Frequentist method, assumes data exchangeability and offers only marginal coverage guarantees [49], [50], Bayesian approaches provide more flexible uncertainty estimation. However, full Bayesian networks are computationally prohibitive. Instead, we adopt Bayesian approximations, specifically Monte Carlo Dropout and Variational Inference, which deliver instance-level uncertainty efficiently and integrate well with existing models.

To answer the question of which UQ approaches are suitable for the specific domain of vulnerability patch datasets, we first introduce three UQ approximation techniques in Section III-A. Following that, Section III-B discusses two different assumptions underpinning the representation of vulnerability data, specifically whether it is homoscedastic or heteroscedastic. Subsequently, we examine metrics that can effectively gauge the accuracy of the measured uncertainty in Section III-C. Finally, we discuss the evaluation experiment setup and the results in Section III-D and Section III-E, respectively.

A. UQ Approximation Techniques

Bayesian neural networks provide a probabilistic framework for UQ, where we have a natural representation of uncertainty, i.e., the probability distribution $p(y|x, D, \theta)$ where D represents the training data, θ the model parameters, x a data instance, and y the predicted label. Because Bayesian neural networks are computationally expensive [51], in practice, we usually approximate Bayesian neural networks through sampling of the weights of the neural networks trained using gradient descent. Two prominent sampling approaches are Monte Carlo Dropout and Model Ensemble [16], [19]. As a baseline commonly used in the literature, we also include the Vanilla UQ model that is derived directly from the training of the neural network [18].

1) *Vanilla*: Deep learning models targeting software engineering applications, e.g., CodeBERT, PLBART, and CommitBART [52]–[54], are multi-layer neural networks commonly trained with gradient descent, which can offer a prediction confidence as output. The prediction confidence is a point estimate, e.g., as $p(y|x, D) = \max_{\theta} p(y|x, D, \theta)$, which is often treated as a simple and straightforward UQ solution [55]–[57].

2) *Monte Carlo Dropout*: Monte Carlo Dropout is a popular regularization technique used in training neural networks,

where a neural network layer’s output is connected to a dropout layer. With it, we drop, i.e., zero out, the activation of the connected neurons at probability P_d during training. When dropouts are used for UQ, we also turn on dropout during inference time, which provides a sampling of a single set of neural network weights [16]. In essence, with Monte Carlo Dropout, we approximate probability distribution $p(y|x, D, \theta)$ with a discrete probability distribution, i.e., $p(y|x, D, \theta_i)$, $i = 0, 1, 2, \dots, T - 1$ by sampling multiple sets of neural network weights with T stochastic passes.

3) *Model Ensemble*: Model Ensemble is another method to sample neural network parameters. It begins with an ensemble, a set of neural networks that are, independently, initialized with their parameters (network weights) and trained. A T ensemble of neural networks would provide T samples of network parameters. This ensemble gives us $p(y|x, D, \theta_i)$, $i = 0, 1, 2, \dots, T - 1$, which serves as an approximation to $p(y|x, D, \theta)$. This method requires neither modification of the neural network nor the presence of dropout layers in an existing model. However, it does impose a greater training cost when compared to the Monte Carlo Dropout approach.

B. Homoscedastic vs. Heteroscedastic Models

Next, we explore another dimension: how we model the distribution of noise in the changeset data, where each instance is represented by a set of features computed from the changesets. We examine two modeling choices: homoscedastic and heteroscedastic models.

1) *Homoscedastic Models*: In homoscedastic models, we assume all changeset instances are identically distributed, e.g., as a Gaussian distribution with a common mean and variance for all changesets. The classification model is therefore homoscedastic as the variance is constant across all changesets. We can create a homoscedastic model as follows: given the feature representations of changesets, a multi-layer perceptron (Figure 2) classifies changesets into one of two classes: vulnerability or non-vulnerability patch. The output layer has two neurons representing the outputs for these two classes. An output is commonly called a logit (denoted as z). When passing a logit through the softmax function, i.e., $S(\cdot)$, we obtain a discrete probability distribution over the two classes, denoted as $p(y|x)$ where x is the feature vector and y the random variable for the label.

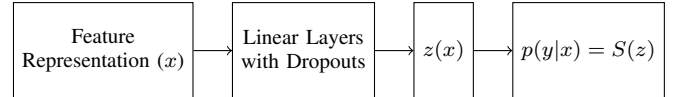


Fig. 2. Design of the homoscedastic model, where $z \in \mathbb{R}^N$, and $N = 2$.

2) *Heteroscedastic Models*: Homoscedastic models’ assumption that changesets are identically distributed is unlikely to hold. For instance, changesets from different software projects or those from different development phases of a single project are likely to have different characteristics. This may result in significantly different data distribution in (the specific features of interest of) different changesets. Heteroscedastic

models although more complex may be more suitable for changeset data.

In this paper, we realize heteroscedastic models via multi-output-head neural networks [17]. We illustrate the model in Figure 3. For changeset x , we assume the output logit follows a Gaussian distribution, i.e., $z \sim \mathcal{N}(\mu, \sigma^2)$ (as shown in Figure 3). This is a model with dual output head, one representing μ and the other σ . As a heteroscedastic model, given N changesets, we obtain N Gaussian distributions.

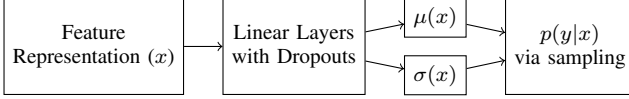


Fig. 3. Design of the heteroscedastic model. It has two output heads for $\mu \in \mathbb{R}^n$ and $\sigma \in \mathbb{R}^{+n}$, where $n = 2$ is the number of classes.

C. Uncertainty Measures

Uncertainty is naturally captured as a probability distribution $p(y|x, D, \theta)$. However, because it can be challenging to compare probability distributions to gauge different level of uncertainty, we sometimes desire a “one-number” quantity to indicate the uncertainty of our prediction although such a quantity do not fully capture it.

Two frequently referenced uncertainty quantities are predictive entropy and mutual information [58]–[60]. Via T samples on model weights w , e.g., from T stochastic passes via Monte Carlo Dropout or from T models via Model Ensemble, we estimate $p(y|x, D) \approx \bar{p}(y|x, D) = \frac{1}{T} \sum_{\theta} p(y|x, D, \theta)$. Then we compute predictive entropy as:

$$\mathcal{H}(y|x, D) = - \sum_c p(y|x, D) \log p(y|x, D) \quad (1)$$

and mutual information as:

$$\mathcal{I}(y|x, D) = \mathcal{H}(y|x, D) - \mathbb{E}_{p(\theta|D)} \mathcal{H}(y|x, \theta) \quad (2)$$

where:

$$\begin{aligned} & \mathbb{E}_{p(\theta|D)} \mathcal{H}(y|x, \theta) \\ &= - \frac{1}{T} \sum_c \sum_{t=1}^T p(y = c|x, D, \theta_t) \log p(y = c|x, D, \theta_t) \end{aligned} \quad (3)$$

where c is for all possible classes, D the set of training data, and θ the model parameters.

There is often a need to disentangle aleatoric and epistemic uncertainties [39], [58], as this work also demonstrates. The Law of Total Variance states that total variance can be decomposed into two components,

$$Var(y) = E[Var[y|x]] + Var[E[y|x]] \quad (4)$$

where, in this study, x represents a *single* commit patch and y its vulnerability status. A patch identification model, a neural network, approximates the function $y = \phi(x)$. We treat $Var(y)$ as a representation of total uncertainty. Since $E[y|x]$ is expected to average out the noise, the variance of $E[y|x]$

TABLE II
SUMMARY OF THE EXPERIMENTAL CONDITIONS.

Design Factor	Values
Data Modeling	Homoscedastic; Heteroscedastic
UQ Approximation	Vanilla; Monte Carlo (MC) Dropout; Model Ensemble
Datasets	SAP [33]; VCMATCH [14]
Data Features	Embeddings (using CodeBERT [52]); Manually-crafted features (from Patch-Scout [13] and VCMATCH [14])

can be attributed to the model’s lack of knowledge during inference. As such, we consider $Var[E[y|x]]$ an estimate of epistemic uncertainty. Subtracting the epistemic component leaves the aleatoric uncertainty. Accordingly, we regard $E[Var[y|x]]$ as an estimate of aleatoric uncertainty. Given this understanding, in the context of homoscedastic models, the literature typically refers to the entropy defined in equation (1) as a measure of total uncertainty, the quality defined in equation (3) as a measure of data or aleatoric uncertainty, and the mutual information estimated in equation (2) as a measure of model or epistemic uncertainty [58]–[60]. In the context of heteroscedastic models, we obtain two probabilistic distributions, one representing aleatoric uncertainty, $p_{ale}(y|x)$, estimated from the expectation of σ , and the other representing epistemic uncertainty, $p_{epi}(y|x)$, computed using the variance of μ (see Figure 3). With these two distributions, we can compute two entropy values, denoted as $\mathcal{H}_{ale}(y|x)$ and $\mathcal{H}_{epi}(y|x)$ according to equation (1), which correspond to aleatoric and epistemic uncertainties, respectively. A detailed explanation of the application of the Law of Total Variance to separate total uncertainty into epistemic and aleatoric uncertainty can be found in the machine learning literature [20], [39], [58].

D. Experiment Setup

Table II provides a summary of the conditions for the experiment. In total, we utilize three UQ approximation techniques: Vanilla, Monte Carlo Dropout, and Model Ensemble (Section III-A). These three techniques are examined alongside two data modeling approaches, homoscedastic and heteroscedastic (Section III-B), resulting in six design combinations for our data curation and quality ranking algorithms. To mitigate threats to the validity of the study, we apply these techniques to two independently curated datasets using two separate feature extraction methods. In the following text, we describe the experimental setup, including the datasets, data features, and procedures.

1) *Quality Metrics for UQ*: In order to evaluate the quality of the proposed UQ techniques, we rely on the Brier Score metric, which is commonly used for this purpose [18], [19], [38], [61], and on F1-Score, which is a popular classification metric:

Brier Score (BS) is defined as $BS(F, Y) = \frac{1}{N} \sum_{t=1}^N \sum_{i=1}^{N_c} (p_{ti} - y_{ti})^2$. For classification tasks, given data instance $x_t \in X$, while y_{ti} is the i -th element of one-hot encoded label of $y_t \in Y$, p_{ti} is the predicted probability for

class i according to classifier F . Brier Score is a strictly proper scoring rule, i.e., a *more accurate model always produces a smaller score*.

F1-Score is a standard classification evaluation metric. It is defined as the harmonic mean of Precision and Recall and can be computed as $2N_{TP}/(2N_{TP} + N_{FP} + N_{FN})$, where N_{TP} represents the number of true positives, N_{FP} the number of false positives, and N_{FN} the number of false negatives. Because changesets are highly class imbalanced, i.e., vulnerability patches are a small minority among all patches, we choose the F1-Score for evaluation.

2) *Datasets and Feature Representation*: We use two manually-validated datasets, VCMATCH and SAP. Although these datasets are relatively smaller in size, they are known for their high quality, which is crucial for the nature of our experiment. High-quality data is essential to ensure that the evaluation is not contaminated by the noise commonly present in many vulnerability patch datasets, as reported in numerous prior studies [3], [4], [13], [14], [34]–[37]. The VCMATCH dataset contains 1,669 researcher-validated vulnerability patches and matching CVEs from 10 OSS projects (FFmpeg, ImageMagick, Jenkins, Linux, Moodle, OpenSSL, phpMyAdmin, PHP-src, QEMU, and Wireshark) [14]. The SAP dataset is validated by their production software teams and contains 1,282 patches to publicly disclosed vulnerabilities affecting 205 distinct open-source Java projects referenced by SAP’s software [62]. In the datasets, we need both vulnerability and non-vulnerability patches. For each vulnerability patch, we randomly sample 5,000 non-vulnerability patches from the software projects, similar to the process performed by VCMATCH and PatchScout [13], [14].

We consider two separate feature representations of changesets: 1) manually designed features and 2) embeddings. For the former, we adopt the features proposed in PatchScout [13] and also re-used by VCMATCH [14]. These features are computed using NVD records, CVE and CWE attributes, and changesets commit messages and diffs. PatchScout categorizes these 22 features into 4 broad categories of vulnerability identifier features, vulnerability location features, vulnerability type features, vulnerability description text features. For the embeddings, we rely on a popular state-of-the-art LLM – CodeBERT [52]. In this study, as in many others, we treat the last hidden states of CodeBERT’s neural network as embeddings.

3) *Evaluation Procedures*: We implement six deep learning models by combining three UQ techniques (Section III-A) with two model architectures (Section III-B). First, we develop Vanilla models for both homoscedastic and heteroscedastic noise assumptions (Figures 2 and 3). Each model uses CodeBERT for feature extraction, followed by three fully connected layers with dropout. The homoscedastic model applies softmax, while the heteroscedastic model outputs μ via ReLU and σ via softplus to ensure positivity. We extend these base models to support Monte Carlo Dropout by enabling stochastic dropout during training and inference, and Model Ensemble by training five independent models and averaging their predictions.

We use the Adam optimizer during training. For ho-

moscedastic models, we train with the negative log likelihood loss function (i.e., mathematically, the cross-entropy loss in two-class classification), and for heteroscedastic models, we train with a stochastic negative log likelihood loss function introduced by Kendall and Gal [17].

We tune hyperparameters using a validation dataset and monitor training with early stopping, halting after five epochs of no improvement in validation loss. Each layer contains 300 neurons with a dropout rate of 0.1. The model checkpoint with the lowest validation loss is selected for evaluation.

To answer RQ1, we design and carry out a series of numerical experiments. For each experiment, we follow standard training, validation and testing protocols. Unless otherwise stated, the ratio of training and test datasets is 0.8 : 0.2. From the training dataset, 10% is set aside as a validation dataset. During training, the majority class, i.e., non-vulnerability patches, are under-sampled to have a balanced training dataset, while during evaluation the test dataset is kept as is.

E. Evaluation Results

We present the results organized along different themes focused on providing a holistic answer to RQ1.

1) *Comparison of UQ Techniques*: Among numerous UQ approximation methods in the literature, in this paper, we consider three of the most popular: Vanilla, Model Ensemble, and Monte Carlo Dropouts. In order to examine which of these three methods has the best potential to support patch data curation, we train and test these methods with increased dataset quality shift. The dataset quality shift is an artificially induced quality degradation in the data. To this end, we add Gaussian noise to each of the features by drawing a random sample n from a Gaussian distribution with zero mean and diagonal covariance, i.e., from $\mathcal{N}(0, \Sigma)$ where Σ is the covariance matrix. We let $\Sigma = \sigma I$ where I is the identity matrix, i.e., the added noise is independent for different features in the feature vector x of each changeset. We refer to σ as the shift intensity that we vary. Figure 4 shows the F1-score and Brier Score for increasing dataset shift intensity. The overall trend is that when the quality shift intensity increases, the model’s predictive performance and uncertainty get worse (lower F1-score and higher Brier Score). These observations demonstrate that the UQ models respond appropriately to the degraded quality in the data, i.e., when data quality degrades the models suffer from decreased predictive performance (i.e., F1-score) along with increased prediction uncertainties (i.e., Brier Score). The results in Figure 4 are obtained via heteroscedastic models, while similar experimental results for homoscedastic models show the same trends.

While all three UQ approximations capture the overall trend of the data quality shift, we observe that Model Ensemble generally yields the highest predictive performance (1.5%–12.4% gain on F1 score) and the most accurate UQ estimation ($\sim 1\%$ reduction in Brier Score). These observations are consistent with those obtained from similar experiments in the literature [18], [19], [61], which also show Model Ensemble to be a most robust UQ approximation method that can be leveraged for data curation of software vulnerability patches.

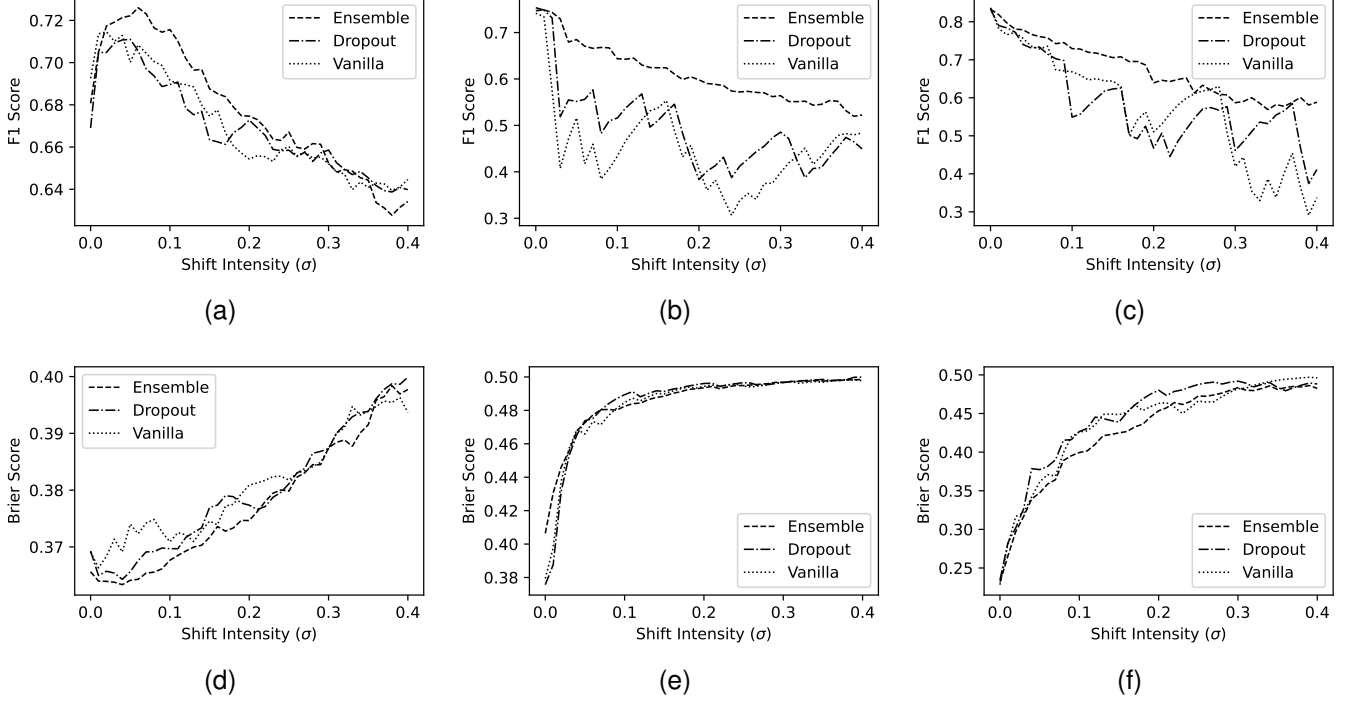


Fig. 4. F1-score and Brier Score vs. dataset quality changes of heteroscedastic models. Subfigures (a) to (c) show the F1-score obtained from three experimental settings: (a) VCMATCH dataset with PatchScout features; (b) VCMATCH dataset with CodeBERT features; and (c) the SAP dataset with CodeBERT features; Subfigures (d) to (f) show the corresponding Brier Score for the three experiments.

Therefore, we conclude that this UQ model is likely to be most beneficial to curate vulnerability patch datasets.

2) *Separating Epistemic from Aleatoric Uncertainty*: When selecting training data for a machine learning model, two key factors are the quality and quantity of data. While high-quality data is preferred, low-quality data can still be useful when no other alternatives are available, as it may help the model gain some understanding of the problem space. Therefore, the decision to include a patch in the dataset should be based on epistemic uncertainty rather than just data quality, i.e., aleatoric uncertainty. A Uncertainty Quantification (UQ) technique that distinguishes between epistemic and aleatoric uncertainties can indicate whether a patch can contribute to the model’s understanding. This is because epistemic uncertainty reflects the model’s knowledge level; high epistemic uncertainty suggests a lack of knowledge about the problem space.

In this section, we aim to ascertain whether the best UQ model we evaluated above, i.e., the heteroscedastic Model Ensemble, can serve as a reliable base for accurately outputting the disentangled uncertainties, i.e., the epistemic and aleatoric uncertainties. We demonstrate that epistemic and aleatoric uncertainties behave as expected when curating patch data. In Table III, we set aside 80% of patches as training data, train the patch identification model with increasing training dataset sizes (60%, 80%, and 100% of the training data), and then estimate the epistemic and aleatoric uncertainties on the test dataset (the remaining 20% of the patches). The table shows that as we increase the training data size, there is a significant reduction in epistemic uncertainty, i.e., a 1.5% to 9.3% decrease in epistemic uncertainty when the training data

TABLE III
EPISTEMIC UNCERTAINTY VS. ALEATORIC UNCERTAINTY AS TRAINING DATA INCREASES

Data	$\overline{\mathcal{H}}_{epi}$	$\Delta_{epi}\%$	$\overline{\mathcal{H}}_{ale}$	$\Delta_{ale}\%$
60%	0.794	0.0%	0.817	0.0%
80%	0.782	$\frac{0.794-0.782}{0.794} \approx 1.5\%$	0.819	$\frac{0.817-0.819}{0.817} \approx -0.2\%$
100%	0.709	$\frac{0.782-0.709}{0.782} \approx 9.3\%$	0.790	$\frac{0.819-0.790}{0.819} \approx 3.5\%$

increased from 60% to 80% and then to 100%. In contrast, the aleatoric uncertainties saw smaller changes. Since an increase in the training data size indicates an improvement in the model’s knowledge about the problem space, which is captured by a reduction in epistemic uncertainty, it follows that for data curation of software vulnerability patches, a UQ technique that disentangles epistemic and aleatoric uncertainties is required.

3) *Heteroscedastic vs. Homoscedastic Models*: Finally, we pose the question of whether modeling the noise in software patch data as heteroscedastic offer any advantage over modeling it as homoscedastic. To optimize computational resources, we focused on the heteroscedastic Model Ensemble, the best-performing UQ approximation, for comparison against homoscedastic UQ techniques. The results in Table IV are obtained by averaging over all data quality shift intensities (from 0 to 0.4 with a step size of 0.1). The results show that the heteroscedastic model offers a significant advantage in predictive performance (higher F1-score) and superior or similar UQ quality (lower or nearly equal Brier Score). Given this observation, we conclude that for vulnerability patches, the UQ measures offered by the heteroscedastic models are

TABLE IV
COMPARISON OF UQ MODELING APPROACHES

UQ Model & Assumption	F1-score	Brier Score
Heteroscedastic Ensemble	0.687 ± 0.017	0.362 ± 0.002
Homoscedastic Ensemble	0.652 ± 0.031	0.395 ± 0.012
Homoscedastic Dropout	0.651 ± 0.029	0.396 ± 0.012
Homoscedastic Vanilla	0.650 ± 0.029	0.396 ± 0.012

more accurate than the homoscedastic models, due likely to the fact that the data distribution in vulnerability patches can vary between projects (and within a project over time), which is more accurately captured by heteroscedastic models.

- 1) From the three UQ models we considered, Model Ensemble produces the most accurate UQ estimation in response to artificial data quality degradation.
- 2) To effectively assess the knowledge gained by a machine learning-based automated patch curation, it is crucial to separate epistemic and aleatoric uncertainty. Epistemic and aleatoric uncertainty are distinguishable during automated patch data curation.
- 3) Heteroscedastic models provide more accurate UQ estimation for patch data curation.

IV. RQ2: CAN WE USE UQ TO IMPROVE AUTOMATICALLY CURATED VULNERABILITY PATCHES BY SELECTING HIGH QUALITY AND HIGHLY USABLE SECURITY PATCHES?

Based on RQ1, we design and evaluate the data curation system. First, we assess the algorithm’s ability to predict patch vulnerability. Second, we examine its impact on vulnerability prediction, a key downstream application (Figure 5).

We begin with a small set of high-quality seed patches (X_s, Y_s) to train a UQ model. Next, we curate patches from a pool X_p of potentially low-quality or project-specific patches. To prevent overlap, we ensure $X_p \cap X_s = \emptyset$. The curation process involves two steps: running inference on X_p and applying the EHAL heuristic to select informative, high-quality patches.

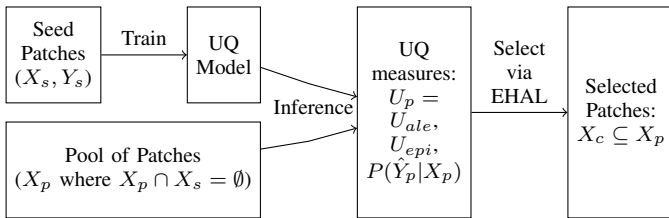


Fig. 5. Evaluation setting for RQ2.

A. Automatic Curation of Vulnerability Patches with UQ

Given an accurate estimation of epistemic and aleatoric uncertainty for patch data curation, as described in RQ1, it is still an open problem of how we use these two quantities to improve patch data curation. In this RQ2, we focus on the practical aspects of using UQ in ML-based patch data curation

in order to obtain higher quality datasets. To this end, we design the EHAL Heuristic (Algorithm 1) that actively and selectively adds patches based primarily on high epistemic uncertainty to include in the dataset. The algorithm relies on a pre-trained UQ model, which takes as input a pool of commit patches and outputs a selected subset. These input patches may be either labeled or unlabeled. For unlabeled patches, the algorithm identifies candidates for annotation, while for labeled ones, it selects instances that are both informative and reliable for training.

Recall that a high epistemic uncertainty of a patch indicates that the model lacks knowledge about this patch, thus the patch is valuable and should be added to the dataset. However, in RQ1 we also observed that a significant degradation of data quality can lead to reduced model performance. Since aleatoric uncertainty is an indicator of the noise in the data, we should avoid those patches that have high aleatoric uncertainty. Therefore, our EHAL algorithm rests on a simple assumption: we should select patches with high epistemic uncertainty, while we should avoid (or not select) those with high aleatoric uncertainty.

Instead of leveraging a formula that combines both epistemic and aleatoric uncertainties, the EHAL heuristic takes a “select-and-then-reject” approach. We opted for this strategy over scalar combinations (e.g., weighted sums of uncertainties) to avoid arbitrary tradeoff calibration. Empirically, we found this approach simpler to tune and less sensitive to outliers in either uncertainty dimension. As shown in Algorithm 1, it first identifies patch instances with the highest epistemic uncertainty as candidates, but rejects them if they are among the patches with the highest aleatoric uncertainty, i.e., it rejects those on the lowest extreme of the quality spectrum. The name of the heuristic is drawn from this idea as well: Epistemic High Aleatoric Low (EHAL).

More specifically, the algorithm’s main function, `DataCurationWithEHAL` (Lines 1–8), curates n patches from a candidate pool $\mathbb{X}_{\text{pool}}$, guided by their pre-computed epistemic and aleatoric uncertainties, $\mathbb{U}_{\text{pool}}$. It iteratively selects patches by invoking the EHAL heuristic (Lines 9–19), which forms the core of the algorithm. The heuristic identifies the candidate patch with the highest epistemic uncertainty using the `TopOneByEpistemicUQ` operation (Lines 12 and 17). Simultaneously, the top n_{ale} patches with the highest aleatoric uncertainty are determined using `TopNByAleatoricUQ` (Lines 13 and 18), where n_{ale} is a hyperparameter determined empirically. If the candidate with the highest epistemic uncertainty is also among the top aleatoric candidates, it is excluded from consideration, and the selection process iterates. This ensures that the final curated dataset includes only patches that are both highly informative and reliable, effectively balancing the trade-off between informativeness (epistemic uncertainty) and quality (low aleatoric uncertainty).

To evaluate the EHAL Heuristic, we consider two baselines. We construct a data selection heuristic completely opposite to the EHAL Heuristic, i.e., Epistemic Low Aleatoric High (ELAH). ELAH selects the patches with the lowest epistemic uncertainty, and rejects them if they are also among those with

Algorithm 1: Patch Curation with EHAL Heuristic

Input: $\mathbb{X}_{\text{pool}}$: Pool of candidate patches
 $\mathbb{U}_{\text{pool}}$: Uncertainty values for candidate patches
 n : Number of patches to curate
 n_{ale} : Hyperparameter for the number of aleatoric instances to consider in each iteration
Output: $\mathbb{D}_{\text{new}}$: Final list of curated patches

```

1 Function DataCurationWithEHAL( $\mathbb{X}_{\text{pool}}, \mathbb{U}_{\text{pool}}, n, n_{\text{ale}}$ )
2    $\mathbb{D}_{\text{new}} \leftarrow []$ 
3   while  $\mathbb{U}_{\text{pool}} \neq \emptyset$  and  $\mathbb{D}_{\text{new}}.\text{size}() < n$  do
4      $d_{\text{epi}} \leftarrow \text{EHAL}(\mathbb{X}_{\text{pool}}, \mathbb{U}_{\text{pool}}, n_{\text{ale}})$ 
5      $\mathbb{X}_{\text{pool}} \leftarrow \mathbb{X}_{\text{pool}} \setminus \{d_{\text{epi}}\}$ 
6      $\mathbb{U}_{\text{pool}} \leftarrow \mathbb{U}_{\text{pool}} \setminus \{d_{\text{epi}}\}$ 
7      $\mathbb{D}_{\text{new}}.\text{append}(d_{\text{epi}})$ 
8   return  $\mathbb{D}_{\text{new}}$ 

9 Function EHAl( $\mathbb{X}_{\text{pool}}, \mathbb{U}_{\text{pool}}, n_{\text{ale}}$ )
10   $\mathbb{X}_{\text{candidate}} \leftarrow \mathbb{X}_{\text{pool}}$ 
11   $\mathbb{U}_{\text{candidate}} \leftarrow \mathbb{U}_{\text{pool}}$ 
12   $d_{\text{epi}} \leftarrow \text{TopOneByEpistemicUQ}(\mathbb{X}_{\text{candidate}}, \mathbb{U}_{\text{candidate}})$ 
13   $\mathbb{D}_{\text{ale}} \leftarrow \text{TopNByAleatoricUQ}(\mathbb{X}_{\text{candidate}}, \mathbb{U}_{\text{candidate}}, n_{\text{ale}})$ 
14  while  $d_{\text{epi}} \in \mathbb{D}_{\text{ale}}$  do
15     $\mathbb{X}_{\text{candidate}} \leftarrow \mathbb{X}_{\text{candidate}} \setminus \{d_{\text{epi}}\}$ 
16     $\mathbb{U}_{\text{candidate}} \leftarrow \mathbb{U}_{\text{candidate}} \setminus \{d_{\text{epi}}\}$ 
17     $d_{\text{epi}} \leftarrow \text{TopOneByEpistemicUQ}(\mathbb{X}_{\text{candidate}}, \mathbb{U}_{\text{candidate}})$ 
18     $\mathbb{D}_{\text{ale}} \leftarrow \text{TopNByAleatoricUQ}(\mathbb{X}_{\text{candidate}}, \mathbb{U}_{\text{candidate}}, n_{\text{ale}})$ 
19  return  $d_{\text{epi}}$ 

```

the lowest aleatoric uncertainty. In addition, we also compare our algorithm with the random baseline that selects patches randomly. Random baselines are frequently used in the active learning literature [63]. By comparing with a random baseline, we can determine whether the data curation algorithm has any knowledge about data selection at all.

In our evaluation, we gradually expand the training dataset, using one of the three algorithms: the proposed one and the two baseline algorithms. For each, we divide the training data partition randomly into two parts: the initial training dataset (20% of available patches) and the candidate training instances pool (60% of available patches). We construct the training dataset, starting with the initial training dataset, and train a patch identification model. With the trained model, we estimate uncertainty measures, such as aleatoric and epistemic uncertainties. Based on the uncertainties, we select 10% of instances from the candidate pool and add these to the training dataset. We then retrain the model and repeat the process. We stop when we exhaust the candidate dataset. To evaluate the model’s predictive performance, we compute the F1 score on the test dataset (20% of available patches) as the size of the training dataset increases. To ensure the reliability of the results, we repeat the process 10 times, each beginning with a random shuffling of the entire dataset.

Figure 6 presents F1-scores obtained by heteroscedastic models using both the VCMATCH dataset with PatchScout features and the SAP dataset with CodeBERT feature representations. There are two primary observations from these results:

a) Improved Patch Curation: The results for both datasets show that the model using EHAl performs the best

(F1 score) as patches are selected and added to the training dataset. The model with the ELAH baseline performs the worst, while random selection is in the middle. This serves as evidence that the proposed algorithm can effectively select patches and improve predictive performance. Note, however, that as the percentage of added training data approaches 100%, there is less leeway for selection, i.e., the dataset runs out of “good” candidates, and adding more patches to the training dataset has no discernible effect. The modest gains observed in Figure 6 are partially due to the limitations of the SAP and VCMATCH datasets, which contain only 1,200 to 1,700 instances. With datasets of this size, it is challenging to observe a large effect.

b) Better Information Efficiency: As shown in Figure 6, the model trained using the VCMATCH dataset reaches the best predictive performance with the EHAl Heuristic (Algorithm 1) when 40% of the patches are added to the training dataset. Since the training time is proportional to the size of the training data, the implication is that by selecting the right patches for the training dataset, we can significantly reduce the training time, i.e., we only need to train with 40% of the data to achieve the best model performance. We observe the similar results regarding the model trained using the SAP data, although the model reaches the best performance when 80% of the data is used for training (Figure 6). This also indicates the SAP datasets is of higher quality as it contains more high quality instances.

The training data ratio is an important hyperparameter in this approach, as it influences the quality of the data curation model. Its optimal value varies depending on the dataset and the model. Our experiments show that, for the VCMATCH dataset, performance gains plateau after using approximately 40% of the curated data, whereas for the higher-quality SAP dataset, 80% was needed to reach maximum utility. This variability underscores the importance of tailoring the method to specific use cases, since the balance between data quality, model performance, and training-time efficiency differs across real-world applications. The data curation process is inherently iterative. A practical approach is to start with a small random sample of labeled patches, estimate a suitable ratio, and then adjust it based on model performance as more data are added.

The evaluation results indicate that the data curation heuristic EHAl can effectively select patches that are high-quality and have high utility values. Using the selected patches results in improvements in the model for automatic vulnerability patch data curation with less training data.

B. Evaluating Patch Curation’s Impact on Automated Software Vulnerability Prediction

In this section, we investigate whether our automatic patch curation algorithm, which uses the EHAl heuristic, can improve software vulnerability prediction [64]. This demonstrates the algorithm’s ability to select high-quality patches, thereby enhancing downstream applications and specifically improving existing vulnerability prediction models.

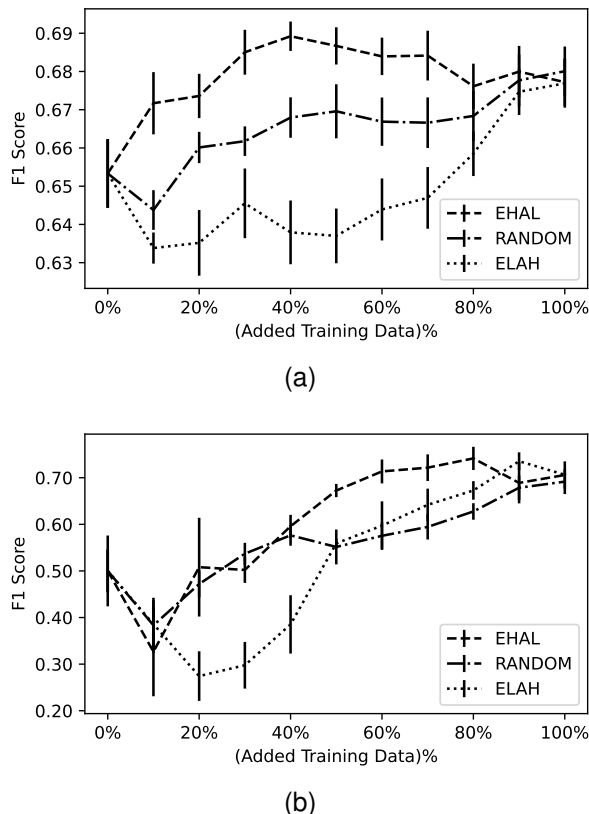


Fig. 6. Comparison of three algorithms/heuristics for selecting training data: Epistemic High Aleatoric Low (EHAL), Epistemic Low Aleatoric High (ELAH), and random choice of data. We compute the F1-score on the: a) VCMATCH dataset with PatchScout features and b) SAP dataset with CodeBERT embeddings. The error bars represent the variance from 10 independent runs while the line represents the average.

Automated software vulnerability prediction is a popular application area that relies on vulnerability patch data. For our experiment, we selected the LineVul vulnerability prediction algorithm [65]. This choice is based on three key considerations. First, LineVul is a state-of-the-art model for predicting both vulnerable functions and vulnerability-inducing lines of code [65]. Second, it was recently evaluated as one of the best-performing models among various vulnerability prediction approaches [66]. Third, it utilizes the Big-Vul dataset [27], a vulnerability dataset where each record contains either a vulnerable or non-vulnerable function. This dataset is widely used because it provides the original patches from which vulnerable functions are derived, allowing for the inference of lines deleted and added before and after security fixes, thus facilitating line-level vulnerability prediction. Additionally, the file-level patches in the Big-Vul dataset allow for assembling commit-level software patches, which our algorithm is designed to validate using UQ.

For this experiment, we apply the EHAL heuristic to select a subset of data from the training dataset, which we use to train the LineVul model. We then evaluate LineVul to assess two dimensions of model performance: predictive performance and computational time. For predictive performance, we examine the F1 score, as done in the LineVul study, while for computational time, we measure the training duration. Our aim is to demonstrate that our UQ-based approach can select high-

quality data that improves LineVul’s predictive performance and reduces its training time.

More specifically, we carry out the experiment according to the following set of steps:

- 1) *Train the patch curation model.* We train a patch curation model using the VCMATCH dataset and select the model that demonstrates the highest patch identification performance within the VCMATCH dataset, leveraging 40% of the training instances (see Figure 6). This is because both the VCMATCH dataset and the Big-Vul dataset consist of C/C++ code, while the SAP dataset contains solely Java code.
- 2) *Remove overlapping projects from the Big-Vul dataset.* The Big-Vul dataset contains functions from 310 software projects, among which 8 are also in the VCMATCH dataset. To eliminate the risk of data contamination, we remove these 8 software projects, namely, FFmpeg, QEMU, OpenSSL, WireShark, PHP-SRC, Moodle, ImageMagick, and Linux, from the Big-Vul dataset completely. As a result, the number of functions in the Big-Vul dataset is reduced from 188,636 to 130,449. For convenience, in the remaining discussion of this section, we refer to this filtered version as the Big-Vul dataset.
- 3) *Extract commit patches from the filtered Big-Vul dataset.* We identify commit hashes associated with vulnerable functions in Big-Vul and use these hashes to extract the relevant patches.
- 4) *Partition data for LineVul training, validation, and testing.* To ensure the reliability of the results, we apply the data split method used in 10-fold cross validation to divide the commit patches in the Big-Vul dataset into the train, validation, and testing partitions, and the ratios of the commits in these three partitions are 0.8 : 0.1 : 0.1. This procedure yields 10 distinct splits with 10 unique test partitions, ensuring no overlap of commits among the training, validation, and test partitions.
- 5) *Use the patch curation model to select LineVul training data.* We select patches from the training partition using the EHAL heuristic. To examine the effects of adding the selected security patches, we gradually increase the selected security patches, i.e., by selecting 10%, 20%, 30%, ..., and 100% of the security patches.
- 6) *Train and evaluate LineVul.* In the Big-Vul data, a vulnerable function has a number of correspondent non-vulnerable functions, all of these are indexed by the commit id of the security patch from which the vulnerable function is extracted. This organization of the Big-Vul data allows us to assemble the training data from the selected security patches. We train a LineVul model from scratch using the functions indexed by the commit id of the selected patches, and evaluate the performance of the trained LineVul model on the test partition. The vulnerable functions in the test partition is about 6.1%, and following the LineVul study [65], we report F1 score as the performance metric.

First, we examine the relationship between LineVul’s predictive performance and the amount of training data used. As

a baseline, we run the identical experiments using a random data selection method. Figure 7 summarizes the experimental results where we show F1 score versus the amount of training data used. LineVul’s predictive performance varies given different test data. The figure exhibits the variance using a standard boxplots showing min, first quartile, median, third quartile, and max. The figure shows that (a) when we use the proposed patch selection method, LineVul achieves a higher F1 score with 20-60% of the data than with 100% of the training data, and (b) the proposed method consistently outperforms random selection except the case when nearly 90% or more patches are selected.

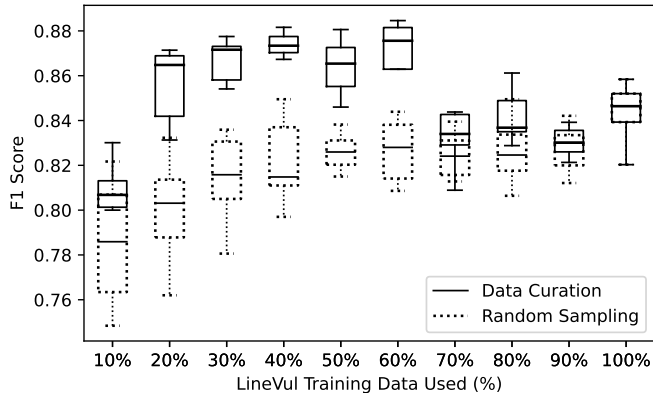


Fig. 7. Performance of LineVul trained using the proposed patch curation method and the random selection method.

Additionally, in Table V we compare the mean F1 scores obtained when using 100% of training data, using the training data selected by the proposed data curation method, and using random selection. When comparing the proposed data curation method with no data curation, i.e., using 100% of training data, the improvement of mean F1 score ranges from 1.66% to 3.52%. We observe a higher improvement of F1 score when comparing the proposed data selection with the random data selection.

TABLE V
COMPARING MEAN F1 SCORES: USING 100% TRAINING DATA (MEAN F1 SCORE=0.843), USING PROPOSED DATA CURATION METHOD, AND RANDOM SELECTION

Selected %	Mean F1 Score		Gain	
	Random	Curation	vs. Random	vs. 100%
20%	0.800	0.857	7.07%	1.66%
30%	0.818	0.868	6.02%	2.94%
40%	0.823	0.872	6.05%	3.52%
50%	0.825	0.858	3.98%	1.81%
60%	0.825	0.860	4.22%	2.08%

Although the improvement is not large, we stress that the improvement is statistically significant. We conduct the Mann-Whitney U test to test the hypothesis that the proposed data curation method yield greater F1 scores despite the variance and the small effective size. The Mann-Whitney U test is a non-parametric test that is suitable for F1 scores, one-sided thus non-Gaussian. Table VI shows that p-value is far less than 5% and we reject the null hypothesis that we obtain superior

F1 scores using the proposed data curation method is due to chance. Given that we control all factors in our experiments, we conclude that the proposed method is superior, as it robustly yields higher F1 scores.

TABLE VI
HYPOTHESIS TESTING VIA MANN-WHITNEY U TEST: PROPOSED DATA CURATION PRODUCES GREATER F1 SCORE THAN USING 100% DATA AND THAN RANDOM SELECTION

Selected %	p-value	
	Curation vs. Random	Curation vs. 100%
20%	$8.23e-05$	$1.43e-24$
30%	$2.68e-04$	$1.63e-06$
40%	$5.95e-06$	$1.84e-07$
50%	$6.15e-04$	$2.27e-03$
60%	$4.45e-03$	$5.17e-03$

Finally, we emphasize that our method can have another benefit, i.e., the reduced computational cost. Machine learning models are increasing in size, thus increased computational cost. To illustrate this benefit, we measure computational time. For selected experiments, we run the experiments in an otherwise idle computer system and record wall-clock time. The computer system is equipped with an AMD EPYC 7742 64-Core Processor, 512 GB RAM, and a TESLA V100S GPU. We compare to a baseline of the LineVul model trained with 100% of its training data.

The experimental result in Table VII shows that the proposed patch curation algorithm, which selects 40% of the training data, can benefit LineVul by gaining a small improvement in predictive performance of 3.52% (Table VI). However, at the same time, the training time is substantially reduced by approximately 56%, i.e., a significant energy saving in training. This benefit is particularly valuable for large vulnerability data collections (that may even include data augmentation), which are increasingly important to modern ML models.

The observed improvement in predictive performance is limited by the fact that the test partition extracted from the Big-Vul data used in the LineVul study is likely to contain some noise, as it was gathered directly using an automated approach based on a set of heuristics and has been reported to contain some labeling errors [67]. Furthermore, the evaluation is limited by the size of the dataset for LineVul, i.e., the Big-Vul dataset.

A trained model of the proposed UQ-based data curation approach based on the EHALL heuristics can improve the accuracy and training time of software vulnerability

TABLE VII
APPLYING PATCH SELECTION TO VULNERABILITY PREDICTION (LINEVUL)

Training Data Selected %	Performance		Computation Time	
	F1	Gain %	Time	Reduction
100%	0.843	—	8h57m	—
40%	0.872	3.52%	3h55m	$\frac{8h57m-3h55m}{8h57m} \approx 56\%$

prediction, a popular use of software vulnerability patch data.

V. RELATED WORK

We divide the related work into two categories: 1) approaches for vulnerability patch curation; and 2) uncertainty quantification.

A. Vulnerability Patch Identification

The research community has been diligently contributing efforts to curate multiple vulnerability patch datasets, and a common method employed by these researchers is through manual reviews [22]–[24], [26]–[28], [30]–[33]. More recently, researchers have started to scale vulnerability patch curation through machine learning, a semi-automated process where they train a machine learning model using a set of manually vetted vulnerability patches and subsequently identify vulnerability patches through inference [3], [4], [13], [14], [34]–[37]. Regarding vulnerability patch curation, we classify these efforts into two distinct categories, which we term as vulnerability patch prediction and vulnerability patch association. The former involves the classification of a changeset, whether it is one commit or a group of semantically related commits, into either a “general” vulnerability changeset (i.e., a changeset that fixes a vulnerability) or not [3], [4], [34]–[37]. On the other hand, the latter aims to determine whether a changeset corresponds to a “specific type” of vulnerability changeset that addresses a known vulnerability, such as a CVE [13], [14].

In this work, we make two essential contributions. First, we provide a principled approach to assess the quality of data curation [3], [4], [34]–[37]. Second, we offer guidance on data curation, focusing on not only data quality but also data usefulness, through the application of Uncertainty Quantification (UQ) techniques [13], [14]. As such, our work is not simply to “clean” existing datasets, rather it is aimed to curate highly useful security patches for improving downstream applications of the patches. While dataset cleaning primarily addresses technical errors and inconsistencies, curation involves selecting data points that maximize utility, ensuring the resulting dataset is optimized for improving model performance and training efficiency.

B. Uncertainty Qualification

In our work, we leverage recent advances in UQ for deep learning. Most UQ techniques are probabilistic approaches [40]–[43]. We can contrast different UQ techniques along several dimensions, e.g., frequentist versus Bayesian approaches, single prediction versus set prediction approaches, direct modeling versus approximation [39], [41]. Primarily, UQ has been leveraged to address “AI safety”, for which, an essential problem is to understand to what extent the user can trust the decision made by a machine learning model [68], [69]. Not until recently have researchers begun to use UQ to assess data quality for machine learning [70]. Our work here is not to propose a novel UQ approach for AI safety. Rather it is to investigate two interconnected issues [15], [37], the data quantity and the data quality of vulnerability

patch data via UQ approaches that disentangle epistemic and aleatoric uncertainties [17], [58]. Via evaluating combinations of data distribution modeling and UQ approximation techniques, our work paves a path for a principled approach to curate and select changesets for software security applications. In addition, our result show that the proposed approach can positively contribute to important research direction in security assurance, such as software vulnerability prediction.

VI. THREATS TO VALIDITY

The primary threat to validity is the inability to generalize the findings, which map to internal threat to validity.

Selection of UQ Models. There are a wealth of UQ research that have resulted in a myriad of UQ concepts and modeling approaches with varying assumptions, advantages, and pitfalls [39], [43]. Selecting a suitable UQ model is a challenging task for software vulnerability patch curation. This challenge is complicated by the lack of ground-truth uncertainty for real-world datasets. To evaluate ordinary machine learning tasks, such as regression and classification, we rely on ground truth values that are sometimes referred to as “gold set” data. For UQ studies on practical problems, there is a lack of UQ gold set data to evaluate UQ models. To address this threat, research commonly experiments with how UQ models respond to data quality shifts. Intuitively, when data quality shifts increases, we expect the models’ prediction ability to degrade, which should result in lower predictive performance and higher uncertainty. In this approach, we experiment with multiple UQ modeling techniques to select the most suitable model for this study.

Generalization of Findings. Our research findings strongly suggest that Uncertainty Quantification (UQ) can serve as a systematic and principled approach to curate software changeset data, enabling effective vulnerability patch identification and other software quality assurance tasks. However, it is important to acknowledge a potential internal threat to the validity of these conclusions, as the generalizability of our results might be limited due to the number of models, datasets, and UQ approaches explored in our initial experiments. Furthermore, we acknowledge that our technique has thus far been applied only to the SAP, VCMATCH, and Big-Vul datasets. To mitigate this internal validity concern, we conducted an extensive array of experiments. By exploring various combinations of UQ techniques, datasets, and data features, we sought to ensure robustness in our findings. The key conclusions consistently emerged across all of our experiments, reinforcing the reliability of our main research outcomes.

This study randomly splits labeled commit patches into training, validation, and test partitions, a standard practice in software vulnerability prediction research. While data leakage is a general concern in machine learning, random splits cannot fully prevent it due to potential dependencies between commits, such as change couplings and genealogies [71], [72]. In our case, this risk is minimal. The datasets consist of vulnerability patches and randomly selected clean patches, without full evolutionary histories. As a result, the chance of leakage from commit dependencies is low.

Sources of Uncertainty. A recent study calls into the attention of the effectiveness of noisy label learning on deep learning for program understanding [73]. The study indicates that deep learning models still struggle to detect real-world noise in program understanding datasets. In this research, we do not differentiate the sources of uncertainty, i.e., whether they stem from the feature space or the labeling space. A future direction will be to investigate whether UQ can improve noisy label learning for program understanding by more accurately detecting labeling errors.

Programming Languages and Data Selection. Programming Languages and Data Selection. In this paper, we evaluated the impact of the proposed data selection approach on software vulnerability prediction using C/C++ code. Whether this data selection approach has a similar impact on software vulnerability prediction for other programming languages or in a cross-language setting remains an open question for future exploration.

VII. CONCLUSION AND FUTURE WORK

Machine learning techniques that leverage historical vulnerabilities have become an important direction in software security assurance. However, obtaining high-quality vulnerability patch data, which is crucial for advancing this research, poses significant challenges. Several studies confirm substantial quality issues in software vulnerability patches collected from the NVD [13], [15]. Furthermore, there is a lack of a systematic approach to curate patch datasets and assess patch quality. Relying solely on software security experts for manual curation is impractical due to cost and limited availability of expertise. In addition, prior works primarily focus on data quality alone [15]. As machine learning techniques become an important research direction software security assurance, whether curated patches are informative to machine learning should be an important dimension to examine. However, designing patch data curation approaches that account for both utility and quality is particularly challenging to design due to the fact that both quality and usefulness span a spectrum.

To address these concerns and challenges, our work aims to develop an approach for curating software changeset data for security software assurance. We propose an automatic, machine learning-based security patch curation approach intended to select patches with high quality and utility value. This is made possible by developing a heuristic that leverages both epistemic and aleatoric uncertainties computed on security patches. From our evaluation, we observe the following: 1) Model Ensemble is an effective method for producing reliable Uncertainty Quantification (UQ) in security patch curation; 2) software changeset data can exhibit different distributions across projects or components, making heteroscedastic modeling a superior approach that demonstrates better predictive performance and UQ quality compared to homoscedastic modeling; 3) epistemic uncertainty can serve as a proxy for the informativeness of the patches while aleatoric uncertainty can serve as a guidance for the quality of the patches; 4) in all evaluations, we compare data curated using the proposed approach with 100% of the data from VCMATCH and BigVul, representing heuristic-based and machine learning-based

state-of-the-art curation methods, respectively, and find that it selects higher-quality and more informative patches. Our study highlights that UQ measures can guide the selection of changesets to benefit downstream secure software quality assurance tasks, such as a software vulnerability prediction model. For a manually curated high-quality dataset, the benefits come 1) a small but statistically significant improvement of predictive performance, and 2) a significant reduction in software vulnerability model training time (i.e., significant energy saving). Our work is a first to curate security patches with a focus on their utility values in addition to their quality, which sets the stage for the future work that aims to scale our experiments to other, larger vulnerability datasets and curate vulnerability patches from databases such as the NVD to significantly improve predictive performance and reduce uncertainty for machine learning-based software security assurance approaches.

Future work also includes assessing new UQ models aimed at enabling a data curation pipeline that further improves downstream applications. Additionally, advances in large language models (LLMs) have led to the exploration of non-training methods, such as in-context learning and prompt engineering, in numerous application settings [74], [75]. A recent study demonstrates the potential of generative LLMs in filtering vulnerability data by identifying and removing low-quality instances using carefully designed prompts [76]. We posit that our work can also be used to curate high-utility, representative examples for prompts or in-context learning inputs, thereby enhancing the effectiveness of these non-training methods.

ACKNOWLEDGEMENTS

This research utilized resources provided by the High Performance Research Computing (HPRC) core facility at Virginia Commonwealth University (<https://hprc.vcu.edu>). Chen and Zhao also acknowledge Lawrence Goetz from the Department of Computer and Information Science at Brooklyn College for maintaining the departmental computational cluster and network infrastructure.

REFERENCES

- [1] G. Altekar, I. Bagrak, P. Burstein, and A. Schultz, "OPUS: online patches and updates for security," in *Proceedings of the 14th conference on USENIX Security Symposium - Volume 14*, ser. SSYM'05. USA: USENIX Association, Jul. 2005, p. 19.
- [2] Y. Chen, Y. Zhang, Z. Wang, L. Xia, C. Bao, and T. Wei, "Adaptive android kernel live patching," in *Proceedings of the 26th USENIX Conference on Security Symposium*, ser. SEC'17. USA: USENIX Association, Aug. 2017, pp. 1253–1270.
- [3] J. Zhou, M. Pacheco, Z. Wan, X. Xia, D. Lo, Y. Wang, and A. E. Hassan, "Finding A Needle in a Haystack: Automated Mining of Silent Vulnerability Fixes," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Nov. 2021, pp. 705–716, iSSN: 2643-1572.
- [4] B. Wu, S. Liu, R. Feng, X. Xie, J. Siow, and S.-W. Lin, "Enhancing Security Patch Identification by Capturing Structures in Commits," *IEEE Transactions on Dependable and Secure Computing*, pp. 1–15, 2022, conference Name: IEEE Transactions on Dependable and Secure Computing.
- [5] J. Jang, A. Agrawal, and D. Brumley, "ReDeBug: Finding Unpatched Code Clones in Entire OS Distributions," in *2012 IEEE Symposium on Security and Privacy*, May 2012, pp. 48–62, iSSN: 2375-1207.

- [6] S. Kim, S. Woo, H. Lee, and H. Oh, "VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery," in *2017 IEEE Symposium on Security and Privacy (SP)*, May 2017, pp. 595–614, iSSN: 2375-1207.
- [7] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, Sep. 2022, conference Name: IEEE Transactions on Software Engineering.
- [8] T. H. M. Le, D. Hin, R. Croft, and M. A. Babar, "DeepCVA: Automated commit-level vulnerability assessment with deep multi-task learning," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2021, pp. 717–729.
- [9] S. M. Ghaffarian and H. R. Shahriari, "Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey," *ACM Computing Surveys*, vol. 50, no. 4, Aug. 2017, place: New York, NY, USA Publisher: Association for Computing Machinery.
- [10] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, and H. Jin, "VulDeeLocator: a deep learning-based fine-grained vulnerability detector," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2821–2837, 2021.
- [11] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "Vuldeepecker: A deep learning-based system for vulnerability detection," *arXiv preprint arXiv:1801.01681*, 2018.
- [12] S. Christey and B. Martin, "Buying into the bias: Why vulnerability statistics suck," *BlackHat, Las Vegas, USA, Tech. Rep.*, vol. 1, pp. 1:1–1:22, 2013, available: https://www.mitre.org/sites/default/files/pdf/martin_cve_paper.pdf, retrieved November 1, 2023.
- [13] X. Tan, Y. Zhang, C. Mi, J. Cao, K. Sun, Y. Lin, and M. Yang, "Locating the Security Patches for Disclosed OSS Vulnerabilities with Vulnerability-Commit Correlation Ranking," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '21, 2021, pp. 3282–3299, event-place: Virtual Event, Republic of Korea.
- [14] S. Wang, Y. Zhang, L. Bao, X. Xia, and M. Wu, "VCMATCH: A ranking-based approach for automatic security patches localization for oss vulnerabilities," in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, IEEE. IEEE, 2022, pp. 589–600.
- [15] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 121–133.
- [16] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *international conference on machine learning*. PMLR, 2016, pp. 1050–1059.
- [17] A. Kendall and Y. Gal, "What uncertainties do we need in bayesian deep learning for computer vision?" in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/2650d6089a6d640c5e85b2b88265dc2b-Paper.pdf
- [18] Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J. Dillon, B. Lakshminarayanan, and J. Snoek, "Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Online: Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/8558cb408c1d76621371888657d2eb1d-Paper.pdf
- [19] R. Rahaman *et al.*, "Uncertainty quantification and deep ensembles," *Advances in Neural Information Processing Systems*, vol. 34, pp. 20063–20075, 2021.
- [20] M. Valdenegro-Toro and D. S. Mori, "A deeper look into aleatoric and epistemic uncertainty disentanglement," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, IEEE. New Orleans, Louisiana, USA: IEEE, 2022, pp. 1508–1516.
- [21] M. Seitzer, A. Tavakoli, D. Antic, and G. Martius, "On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks," *arXiv preprint arXiv:2203.09168*, 2022.
- [22] S. Rei and R. Abreu, "A Database of Existing Vulnerabilities to Enable Controlled Testing Studies," *International Journal of Secure Software Engineering (IJSSSE)*, vol. 8, no. 3, pp. 1–23, Jul. 2017, publisher: IGI Global. [Online]. Available: <https://www.igi-global.com/article/a-database-of-existing-vulnerabilities-to-enable-controlled-testing-studies/www.igi-global.com/article/a-database-of-existing-vulnerabilities-to-enable-controlled-testing-studies/201213>
- [23] G. Lin, S. Wen, Q.-L. Han, J. Zhang, and Y. Xiang, "Software vulnerability detection using deep neural networks: a survey," *Proceedings of the IEEE*, vol. 108, no. 10, pp. 1825–1848, 2020, publisher: IEEE.
- [24] M. Jimenez, Y. Le Traon, and M. Papadakis, "[Engineering Paper] Enabling the Continuous Analysis of Security Vulnerabilities with VulData7," in *2018 IEEE 18th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, Sep. 2018, pp. 56–61, iSSN: 2470-6892.
- [25] A. D. Sawadogo, T. F. Bissyandé, N. Moha, K. Allix, J. Klein, L. Li, and Y. L. Traon, "Learning to Catch Security Patches," Jan. 2020, arXiv:2001.09148 [cs]. [Online]. Available: <http://arxiv.org/abs/2001.09148>
- [26] X. Wang, K. Sun, A. Batcheller, and S. Jajodia, "Detecting "0-Day" Vulnerability: An Empirical Study of Secret Security Patch in OSS," in *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, Jun. 2019, pp. 485–492, iSSN: 1530-0889.
- [27] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries," in *Proceedings of the 17th International Conference on Mining Software Repositories*, ser. MSR '20. New York, NY, USA: Association for Computing Machinery, Sep. 2020, pp. 508–512. [Online]. Available: <https://doi.org/10.1145/3379597.3387501>
- [28] S. Reis and R. Abreu, "A ground-truth dataset of real security patches," Oct. 2021, arXiv:2110.09635 [cs]. [Online]. Available: <http://arxiv.org/abs/2110.09635>
- [29] T. Riom, A. Sawadogo, K. Allix, T. F. Bissyandé, N. Moha, and J. Klein, "Revisiting the VCCFinder approach for the identification of vulnerability-contributing commits," *Empirical Software Engineering*, vol. 26, no. 3, p. 46, Mar. 2021. [Online]. Available: <https://doi.org/10.1007/s10664-021-09944-w>
- [30] G. Bhandari, A. Naseer, and L. Moonen, "CVEfixes: automated collection of vulnerabilities and their fixes from open-source software," in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*, ser. PROMISE 2021. New York, NY, USA: Association for Computing Machinery, Aug. 2021, pp. 30–39. [Online]. Available: <https://doi.org/10.1145/3475960.3475985>
- [31] T. G. Nguyen, T. Le-Cong, H. J. Kang, X.-B. D. Le, and D. Lo, "VulCurator: A Vulnerability-Fixing Commit Detector," Sep. 2022, arXiv:2209.03260 [cs]. [Online]. Available: <http://arxiv.org/abs/2209.03260>
- [32] Vulncode-DB, "<https://www.vulncode-db.com/>," 2022.
- [33] S. E. Ponta, H. Plate, A. Sabetta, M. Bezzi, and C. Dangremont, "A Manually-Curated Dataset of Fixes to Vulnerabilities of Open-Source Software," Mar. 2019, arXiv:1902.02595 [cs]. [Online]. Available: <http://arxiv.org/abs/1902.02595>
- [34] Y. Zhou and A. Sharma, "Automated identification of security issues from commit messages and bug reports," in *Proceedings of the 2017 11th joint meeting on foundations of software engineering*, 2017, pp. 914–919.
- [35] R. Cabrera Lozoya, A. Baumann, A. Sabetta, and M. Bezzi, "Commit2Vec: Learning distributed representations of code changes," *SN Computer Science*, vol. 2, no. 3, p. 150, 2021.
- [36] X. Wang, S. Wang, P. Feng, K. Sun, S. Jajodia, S. Benchaaboun, and F. Geck, "PatchRNN: A deep learning-based system for security patch identification," in *MILCOM 2021-2021 IEEE Military Communications Conference (MILCOM)*. IEEE, 2021, pp. 595–600.
- [37] Y. Zhou, J. K. Siow, C. Wang, S. Liu, and Y. Liu, "SPI: Automated identification of security patches via commits," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 1, pp. 1–27, 2021.
- [38] D. Huseljic, B. Sick, M. Herde, and D. Kottke, "Separation of aleatoric and epistemic uncertainty in deterministic deep neural networks," in *2020 25th International Conference on Pattern Recognition (ICPR)*. IEEE, 2021, pp. 9172–9179.
- [39] E. Hüllermeier and W. Waegeman, "Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods," *Machine Learning*, vol. 110, pp. 457–506, 2021.
- [40] M. Abdar, F. Pourpanah, S. Hussain, D. Rezazadegan, L. Liu, M. Ghavamzadeh, P. Fieguth, X. Cao, A. Khosravi, U. R. Acharya *et al.*, "A review of uncertainty quantification in deep learning: Techniques, applications and challenges," *Information Fusion*, vol. 76, pp. 243–297, 2021.
- [41] J. Gawlikowski, C. R. N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. Kruspe, R. Triebel, P. Jung, R. Roscher *et al.*, "A survey of uncertainty in deep neural networks," *arXiv preprint arXiv:2107.03342*, 2021.
- [42] J. Mena, O. Pujol, and J. Vitria, "A survey on uncertainty estimation in deep learning classification systems from a bayesian perspective," *ACM Computing Surveys (CSUR)*, vol. 54, no. 9, pp. 1–35, 2021.

- [43] W. He and Z. Jiang, "A survey on uncertainty quantification methods for deep neural networks: An uncertainty source perspective," *arXiv preprint arXiv:2302.13425*, 2023.
- [44] R. C. Smith, *Uncertainty Quantification: Theory, Implementation, and Applications*. USA: Society for Industrial and Applied Mathematics, 2013.
- [45] A. Sinha, T. Mickus, M. Clausel, M. Constant, and X. Coubez, "Domain-specific or uncertainty-aware models: Does it really make a difference for biomedical text classification?" in *Proceedings of the 23rd Workshop on Biomedical Natural Language Processing*, 2024, pp. 202–211.
- [46] B. Mucsányi, M. Kirchhof, and S. J. Oh, "Benchmarking uncertainty disentanglement: Specialized uncertainties for specialized tasks," in *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. [Online]. Available: <https://openreview.net/forum?id=x8RgF2xQTj>
- [47] Q. Hu, Y. Guo, X. Xie, M. Cordy, M. Papadakis, L. Ma, and Y. L. Traon, "Codes: Towards code model generalization under distribution shift," in *Proceedings of the 45th International Conference on Software Engineering: New Ideas and Emerging Results*, ser. ICSE-NIER '23. IEEE Press, 2023, p. 1–6. [Online]. Available: <https://doi.org/10.1109/ICSE-NIER58687.2023.00007>
- [48] Y. Li, S. Chen, and W. Yang, "Estimating predictive uncertainty under program data distribution shift," 2021. [Online]. Available: <https://arxiv.org/abs/2107.10989>
- [49] V. Vovk, A. Gammernan, and G. Shafer, *Algorithmic learning in a random world*. Springer Science & Business Media, 2005.
- [50] R. Foygel Barber, E. J. Candes, A. Ramdas, and R. J. Tibshirani, "The limits of distribution-free conditional predictive inference," *Information and Inference: A Journal of the IMA*, vol. 10, no. 2, pp. 455–482, 2021, publisher: Oxford University Press.
- [51] L. V. Jospin, H. Laga, F. Boussaid, W. Buntine, and M. Bennamoun, "Hands-on bayesian neural networks—a tutorial for deep learning users," *IEEE Computational Intelligence Magazine*, vol. 17, no. 2, pp. 29–48, 2022.
- [52] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*. Online: Association for Computational Linguistics, Nov. 2020, pp. 1536–1547. [Online]. Available: <https://aclanthology.org/2020.findings-emnlp.139>
- [53] W. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Online: Association for Computational Linguistics, 2021, pp. 2655–2668.
- [54] S. Liu, Y. Li, X. Xie, and Y. Liu, "CommitBART: A large pre-trained model for github commits," 2023.
- [55] D. Hin, A. Kan, H. Chen, and M. A. Babar, "Linevd: Statement-level vulnerability detection using graph neural networks," in *Proceedings of the 19th international conference on mining software repositories*, 2022, pp. 596–607.
- [56] H. Wang, G. Ye, Z. Tang, S. H. Tan, S. Huang, D. Fang, Y. Feng, L. Bian, and Z. Wang, "Combining graph-based learning with automated data collection for code vulnerability detection," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 1943–1958, 2020.
- [57] X. Wang, S. Wang, P. Feng, K. Sun, and S. Jajodia, "Patchdb: A large-scale security patch dataset," in *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2021, pp. 149–160.
- [58] S. Depeweg, J.-M. Hernandez-Lobato, F. Doshi-Velez, and S. Udluft, "Decomposition of uncertainty in bayesian deep learning for efficient and risk-sensitive learning," in *International Conference on Machine Learning*. PMLR, 2018, pp. 1184–1193.
- [59] A. Malinin and M. Gales, "Predictive uncertainty estimation via prior networks," in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/file/3ea2db50e62ceefceaf70a9d9a56a6f4-Paper.pdf
- [60] L. Smith and Y. Gal, "Understanding measures of uncertainty for adversarial example detection," in *Proceedings of the 2018 Conference on Uncertainty in Artificial Intelligence*, 2018, pp. 207:1–207:10. [Online]. Available: <http://auai.org/uai2018/proceedings/papers/207.pdf>
- [61] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/9ef2ed4b7fd2c810847ffa5fa85bce38-Paper.pdf
- [62] S. E. Ponta, H. Plate, A. Sabetta, M. Bezzi, and C. Dangremont, "A manually-curated dataset of fixes to vulnerabilities of open-source software," in *Proceedings of the 16th International Conference on Mining Software Repositories*, ser. MSR '19. Online: IEEE Press, 2019, p. 383–387. [Online]. Available: <https://doi.org/10.1109/MSR.2019.00064>
- [63] A. Parvaneh, E. Abbasnejad, D. Teney, G. R. Haffari, A. Van Den Hengel, and J. Q. Shi, "Active learning by feature mixing," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 12 237–12 246.
- [64] I. Kalouptoglou, M. Siavvas, A. Ampatzoglou, D. Kehagias, and A. Chatzigeorgiou, "Software vulnerability prediction: A systematic mapping study," *Information and Software Technology*, p. 107303, 2023.
- [65] M. Fu and C. Tantithamthavorn, "Linevul: A transformer-based line-level vulnerability prediction," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 608–620.
- [66] B. Steenhoeck, M. M. Rahman, R. Jiles, and W. Le, "An empirical study of deep learning models for vulnerability detection," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2237–2248.
- [67] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability detection with code language models: How far are we?" *arXiv preprint arXiv:2403.18624*, 2024.
- [68] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in ai safety," *arXiv preprint arXiv:1606.06565*, 2016.
- [69] D. Gros, P. Devanbu, and Z. Yu, "Ai safety subproblems for software engineering researchers," *arXiv preprint arXiv:2304.14597*, 2023.
- [70] N. Seedat, J. Crabbé, I. Bica, and M. van der Schaar, "Data-IQ: Characterizing subgroups with heterogeneous outcomes in tabular data," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 23 660–23 674. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/95b6e2ff961580e03c0a662a63a71812-Paper-Conference.pdf
- [71] M. D'Ambrosio, M. Lanza, and R. Robbes, "On the relationship between change coupling and software defects," in *2009 16th Working Conference on Reverse Engineering*. IEEE, 2009, pp. 135–144.
- [72] K. Herzig, S. Just, A. Rau, and A. Zeller, "Predicting defects using change genealogies," in *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2013, pp. 118–127.
- [73] W. Wang, Y. Li, A. Li, J. Zhang, W. Ma, and Y. Liu, "An empirical study on noisy label learning for program understanding," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–12.
- [74] N. Wies, Y. Levine, and A. Shashua, "The learnability of in-context learning," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 36 637–36 651. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/73950f0eb4ac0925dc71ba2406893320-Paper-Conference.pdf
- [75] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf
- [76] C. Dil, H. Chen, and K. Damevski, "Towards higher quality software vulnerability data using LLM-based patch filtering," *Journal of Systems and Software*, p. 112581, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S016412122500250X>